

SIMATS ENGINEERING (SAVEETHA SCHOOL OF ENGINEERING)

Department of Computer Science and Engineering

ASSESSMENT TOOL 1 – Coding Challenge Task

Course Code:	CSA06	Course Title:	Design and Analysis of Algorithms
CO Assessed:	CO4: Articulation (BL4, BL5)	Assessment:	Assessment Tool 1 – Coding Challenge Task
Weightage:	40% of CIA	Total Marks:	100 (2 Questions × 50 Marks)
CO4 Statement:	Solve optimization problems using dynamic programming and greedy strategies.		
Student Name:	Tejaswi Reddy K	Reg. No.:	192371036
Section / Batch:	CS-BIOSCI	Date:	25-08-2026

Instructions to Students

- Answer both questions. Each question carries 50 Marks. Total: 100 Marks.
- Carefully analyze the given questions.
- Write clear code for the given problem.
- Analyze the Time complexity and Space complexity of your code using dynamic programming and greedy strategies

Question Type	Description	Questions	Total Marks
Coding Challenge Task	Focus on Code and analyze optimization problems using dynamic programming and greedy strategies	Q1 – Q2	100

RUBRICS FOR CALCULATION

Coding Challenge Task

Criteria	Weightage	Level 4 (Exemplary)	Level 3 (Proficient)	Level 2 (Developing)	Level 1 (Beginning)
Problem Understanding	20%	Clearly understands problem constraints, edge cases, and competitive requirements	Understands problem with minor gaps in constraints	Partial understanding; misses key constraints	Misinterprets problem or constraints
Algorithm	25%	Selects optimal algorithm with best time and space complexity for competitive constraints	Uses efficient algorithm with minor scope for improvement	Uses basic/inefficient approach	No clear algorithmic approach
Code Implementation	20%	Writes correct, efficient, and clean code that passes all constraints	Code works with minor errors or inefficiencies	Partially correct code; fails for some cases	Code does not execute or gives incorrect results
Competitive Performance	20%	Solves problem within time limits and handles large inputs effectively	Solves within acceptable time with minor delays	Struggles with time limits or larger inputs	Unable to solve within time constraints
Test Case Handling	15%	Handles all test cases including edge and hidden cases	Handles most test cases	Misses important edge cases	Fails on multiple test cases

SECTION A – Coding Challenge Task

The screenshot displays the SIMATS Online Programming Lab interface. The top navigation bar includes the SIMATS Engineering logo, the lab name, and the user's name 'KOMMURU TEJASWI REDDY'. The main content area is titled 'CO4-AT1-Coding Challenge' and lists two questions. The first question, 'Q1: Dynamic Programming', is selected and shows a problem description for 'Minimum Number of Coins (Unbounded Knapsack)'. The problem asks to determine the minimum number of coins required to reach a target amount using given denominations. The input format specifies the number of coins, the denominations, and the target amount. A sample input is provided: n=3, coins=[1, 3, 4], amount=6, with an expected output of 2. The 'Your Code' section contains a Python solution using dynamic programming. The 'Input' field shows the sample input values. The 'Output' field displays the result 'Min Coins = 2'. A 'Submitted' button is visible at the bottom of the code editor.

CO4-AT1-Coding Challenge

Language: **Python** C C++ Java

Q1: Dynamic Programming 30 marks

Minimum Number of Coins (Unbounded Knapsack)

Given coin denominations and a target value, determine the minimum number of coins required where each coin can be used multiple times. The objective is to reach the exact amount with minimum coins. Use Dynamic Programming to compute the solution efficiently.

Input Format:
n = number of coins
coins[] = denominations
amount = target

Sample Input:
n = 3
coins = 1 3 4
amount = 6
Expected Output:
Min Coins = 2

Your Code

```
def min_coins(n, coins, amount):  
    # dp[i] represents minimum coins needed for amount i  
    dp = [float('inf')] * (amount + 1)  
    dp[0] = 0 # base case: 0 coins needed for amount 0  
  
    for i in range(1, amount + 1):  
        for coin in coins:  
            if coin <= i:  
                dp[i] = min(dp[i], dp[i - coin] + 1)  
  
    return dp[amount] if dp[amount] != float('inf') else -1  
  
# Reading input  
n = int(input())  
coins = list(map(int, input().split()))  
amount = int(input())  
  
result = min_coins(n, coins, amount)  
print("Min Coins = ", result)
```

Input

```
3  
1 3 4  
6
```

Output

```
Min Coins = 2
```

Submitted

© 2026 SIMATS Online Lab. All rights reserved.

CSA621-CO4-AT1-Coding Ch...SIMATS Online Lab

Ask Gemini

Not secure simatslab.saveetha.com/practice_app/classactivity/attempt/assignment_id=1543

SIMATS Online Programming Lab
OAA PROGRAMMING

KOMMURU TEJASWI REDDY
1923702440

CO4-AT1-Coding Challenge

← Back

Language: Python C C++ Java

QUESTIONS

Q1
Dynamic Programming
30 marks

Q2
Dynamic Programming
30 marks

3/3 answered

Q1 Dynamic Programming 30 marks

Longest Bitonic Subsequence

Given an array of integers, find the length of the longest subsequence that first increases and then decreases. The subsequence must follow the original order. The goal is to maximize the length of such a bitonic sequence. Use Dynamic Programming to compute the result.

Input Format:
n = number of elements
arr[] = elements
Sample Input:
n = 6
arr = 1 11 2 10 4 5
Expected Output:
length = 4

Your Code

```
1 def longest_bitonic_subsequence(n, arr):
2     # lis[i] = length of longest increasing subsequence ending at i
3     lis = [1] * n
4     for i in range(1, n):
5         for j in range(i):
6             if arr[i] > arr[j]:
7                 lis[i] = max(lis[i], lis[j] + 1)
8
9     # lds[i] = length of longest decreasing subsequence starting at i
10    lds = [1] * n
11    for i in range(n-2, -1, -1):
12        for j in range(i+1, n):
13            if arr[i] > arr[j]:
14                lds[i] = max(lds[i], lds[j] + 1)
15
16    # Maximum value of (lis[i] + lds[i] - 1) gives the longest bitonic
17    max_length = 0
18    for i in range(n):
19        max_length = max(max_length, lis[i] + lds[i] - 1)
20
21    return max_length
22
23 # Input
24 n = int(input())
25 arr = list(map(int, input().split()))
26
27 # Output
28 print("length =", longest_bitonic_subsequence(n, arr))
```

Input

5
1 11 2 10 4 5

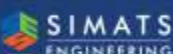
Output

length = 4

CSA621-CO4-AT1-Coding Ch...SIMATS Online Lab

Ask Gemini

Not secure simatslab.saveetha.com/practice_app/classactivity/attempt/assignment_id=1543

SIMATS Online Programming Lab
OAA PROGRAMMING

KOMMURU TEJASWI REDDY
1923702440

CO4-AT1-Coding Challenge

← Back

Language: Python C C++ Java

QUESTIONS

Q1
Dynamic Programming
30 marks

Q2
Dynamic Programming
30 marks

3/3 answered

Q1 Dynamic Programming 30 marks

Longest Bitonic Subsequence

Given an array of integers, find the length of the longest subsequence that first increases and then decreases. The subsequence must follow the original order. The goal is to maximize the length of such a bitonic sequence. Use Dynamic Programming to compute the result.

Input Format:
n = number of elements
arr[] = elements
Sample Input:
n = 6
arr = 1 11 2 10 4 5
Expected Output:
length = 4

Your Code

```
1 def longest_bitonic_subsequence(n, arr):
2     # lis[i] = length of longest increasing subsequence ending at i
3     lis = [1] * n
4     for i in range(1, n):
5         for j in range(i):
6             if arr[i] > arr[j]:
7                 lis[i] = max(lis[i], lis[j] + 1)
8
9     # lds[i] = length of longest decreasing subsequence starting at i
10    lds = [1] * n
11    for i in range(n-2, -1, -1):
12        for j in range(i+1, n):
13            if arr[i] > arr[j]:
14                lds[i] = max(lds[i], lds[j] + 1)
15
16    # Maximum value of (lis[i] + lds[i] - 1) gives the longest bitonic
17    max_length = 0
18    for i in range(n):
19        max_length = max(max_length, lis[i] + lds[i] - 1)
20
21    return max_length
22
23 # Input
24 n = int(input())
25 arr = list(map(int, input().split()))
26
27 # Output
28 print("length =", longest_bitonic_subsequence(n, arr))
```

Input

5
1 11 2 10 4 5

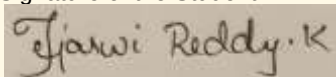
Output

length = 4

Code of Conduct

I Tejaswi Reddy K(192371036) certify that this submission is my original work and that I have adhered to the guidelines specified for this assessment. I understand that any violation of academic integrity rules will result in disciplinary action.

Signature of the Student:

**Mark Evaluation:**

Questions	Problem Understanding (20)	Algorithm (25)	Code Implementation (20)	Competitive Performance (20)	Test Case Handling (15)	Total Marks (Each - 50)
Q1						
Q2						
Overall Total						

Faculty In-charge	Course Coordinator	Head of Department
Name: _____ Signature: _____	Signature: _____	Signature: _____
Date: _____	Date: _____	Date: _____